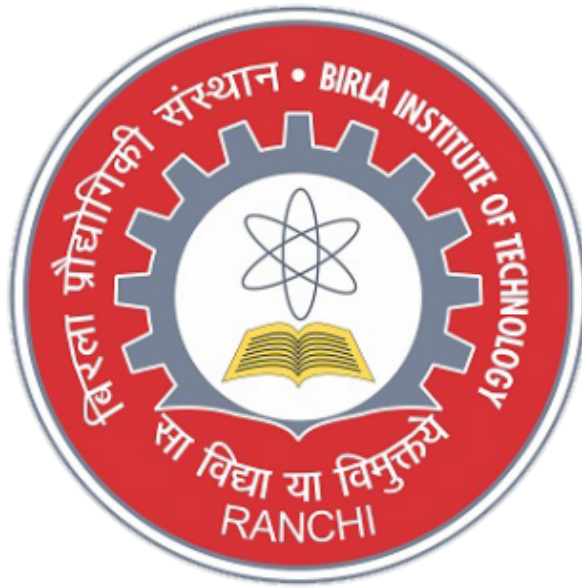




A Comparative Study of Clustering Algorithms for Unsupervised Image Segmentation on the Berkeley Segmentation Dataset (BSDS)

PROJECT REPORT

Submitted by:

Priyanshu Ranjan

Roll No.: IED/10011/23

Course:

Statistical Machine Learning

II

Supervised by:

Dr. Manish K Pandey

Assistant Professor

Institution:

BIT Mesra

Ranchi, Jharkhand

Contents

1	Problem Statement	2
2	Description of the Dataset	2
2.1	Feature Representation	3
3	Whole Code	3
4	Number of Use Cases and Results	8
4.1	Use Cases	8
4.2	Quantitative Results	8
4.3	Visual Results	9
5	Failure Scenarios	10
6	Conclusion	12

1 Problem Statement

Clustering is one of the most widely used unsupervised learning tools for partitioning image pixels into visually and semantically meaningful groups, a task commonly referred to as **image segmentation**. A large number of clustering algorithms exist – partition-based, density-based, hierarchical, and tree-based – and each makes different assumptions about cluster shape, density, and scale. Consequently, the same image can be segmented very differently depending on the algorithm chosen, and there is no single algorithm that performs best on every image.

The objective of this project is to design a controlled experimental pipeline that:

1. Converts a natural image into a pixel-level feature space that combines colour and spatial information.
2. Applies **ten** different clustering algorithms/variants – K-Means, K-Medoids, DBSCAN, DIANA (Divisive Analysis), Agglomerative Hierarchical Clustering (single, complete, average, ward, and centroid linkages), and BIRCH – to the same feature space.
3. Evaluates and compares the resulting segmentations using quantitative internal cluster-validity metrics (Silhouette score, Davies–Bouldin index, and Calinski–Harabasz index) as well as wall-clock running time.
4. Visually inspects the segmented outputs to identify cases where an algorithm succeeds or fails to produce a meaningful segmentation.

The end goal is to determine which clustering strategies are best suited for pixel-level image segmentation on natural photographs, and to understand the practical failure modes of each approach.

2 Description of the Dataset

The project uses the **Berkeley Segmentation Dataset (BSDS)**, a standard benchmark dataset for image segmentation research consisting of natural photographs (landscapes, animals, people, and objects) along with human-annotated ground-truth segmentations.

- **Source folder used:** images/train (a local copy of the BSDS archive).
- **Number of images available:** 201 training images.
- **Image format:** .jpg, RGB natural photographs of varying content (animals, scenery, everyday objects).
- **Sample image used for the experiment:** 100075.jpg (first image in the sorted file listing) – a photograph of a bear standing near water, shown in

Figure 1.

- **Pre-processing:** Each image is converted to RGB and resized to 32×32 pixels. The small resolution is a deliberate design choice because several of the algorithms implemented from scratch (K-Medoids, DIANA) and the pairwise distance matrix computation are $O(n^2)$ – $O(n^3)$ in the number of pixels n , so a small, fixed-size image keeps every algorithm computationally tractable while still preserving the overall structure of the scene.

Original (resized): 100075.jpg

**Figure 1:** Sample image from the BSDS training set (100075 . jpg), resized to 32×32 pixels.

2.1 Feature Representation

Every pixel is represented as a 5-dimensional feature vector combining **colour** and **spatial position**, so that spatially close pixels of similar colour are pulled into the same cluster:

$$\mathbf{x}_i = \left[\underbrace{r_i, g_i, b_i}_{\text{colour, normalised to } [0,1]}, \underbrace{w \cdot \frac{y_i}{h}, w \cdot \frac{x_i}{w_{\text{img}}}}_{\text{spatial coordinates, weighted}} \right]$$

where $w = 0.4$ is the SPATIAL_WEIGHT that controls how strongly pixel position (relative to colour) influences the clustering. For the 32×32 sample image this yields a feature matrix of shape $(1024, 5)$. The target number of clusters/segments is fixed at $K = 6$ for every algorithm that accepts K as an input.

3 Whole Code

The complete, unmodified analysis pipeline (data loading, feature engineering, the ten clustering routines, evaluation, and visualisation) is reproduced below in full.

Listing 1: Complete source code of the clustering-comparison pipeline.

```

1 # --- Setup ---
2 # If any of these are missing, uncomment and run:
3 # !pip install scikit-learn scipy matplotlib numpy pillow pandas --quiet
4

```

```

5 import os, tarfile, time
6 import numpy as np
7 import pandas as pd
8 import matplotlib.pyplot as plt
9 from PIL import Image
10
11 from sklearn.cluster import KMeans, DBSCAN, Birch, AgglomerativeClustering
12 from sklearn.metrics import silhouette_score, davies_bouldin_score,
    calinski_harabasz_score
13 from sklearn.metrics.pairwise import pairwise_distances
14 from scipy.cluster.hierarchy import linkage, fcluster
15 from scipy.spatial.distance import squareform
16
17 np.random.seed(42)
18
19
20 # --- Extract the BSDS300 archive ---
21 ARCHIVE_DIR = "." # folder containing the .tgz files
22 EXTRACT_DIR = "./bsds300_extracted" # where files will be extracted
23
24 os.makedirs(EXTRACT_DIR, exist_ok=True)
25
26 images_tgz = os.path.join(ARCHIVE_DIR, "BSDS300-images.tgz")
27 with tarfile.open(images_tgz, "r:") as tf: # plain tar, not gzip-compressed
28     tf.extractall(EXTRACT_DIR)
29
30 IMAGES_DIR = os.path.join(EXTRACT_DIR, "BSDS300", "images", "train")
31 image_files = sorted(os.listdir(IMAGES_DIR))
32 print(f"Found {len(image_files)} training images. Example: {image_files[:5]}")
33
34
35 # --- Choose an image & build feature matrix [R, G, B, x, y] ---
36 IMAGE_NAME = image_files[0] # change this to try a different image
37 RESIZE_TO = (32, 32) # (width, height) - keep small, DIANA/K-Medoids are  $O(n^2)$ - $O(n^3)$ 
38 SPATIAL_WEIGHT = 0.4 # weight on (x, y) relative to normalized color
39 K = 6 # target number of clusters/segments
40
41 img_path = os.path.join(IMAGES_DIR, IMAGE_NAME)
42 img = Image.open(img_path).convert("RGB").resize(RESIZE_TO, Image.BILINEAR)
43 img_arr = np.array(img)
44 h, w, _ = img_arr.shape
45
46 def image_to_features(img_arr, spatial_weight=0.4):
47     h, w, _ = img_arr.shape
48     color = img_arr.reshape(-1, 3).astype(float) / 255.0
49     yy, xx = np.mgrid[0:h, 0:w]
50     coords = np.stack([yy.ravel() / h, xx.ravel() / w], axis=1) * spatial_weight
51     return np.hstack([color, coords])
52
53 X = image_to_features(img_arr, SPATIAL_WEIGHT)
54 print("Feature matrix shape:", X.shape)
55
56 plt.figure(figsize=(3, 3))
57 plt.imshow(img_arr)
58 plt.title(f"Original (resized): {IMAGE_NAME}")
59 plt.axis("off")
60 plt.show()

```

```

61
62
63 # --- Precompute distance matrix (needed for K-Medoids and DIANA) ---
64 dist_matrix = pairwise_distances(X, metric="euclidean")
65 print("Distance matrix shape:", dist_matrix.shape)
66
67
68 # --- K-Medoids (custom, fast alternating implementation) ---
69 def kmedoids_fit(dist_matrix, k, max_iter=100, random_state=42):
70     n = dist_matrix.shape[0]
71     rng = np.random.RandomState(random_state)
72     medoid_idx = rng.choice(n, k, replace=False)
73     for _ in range(max_iter):
74         labels = np.argmin(dist_matrix[:, medoid_idx], axis=1)
75         new_medoids = medoid_idx.copy()
76         for ci in range(k):
77             members = np.where(labels == ci)[0]
78             if len(members) == 0:
79                 continue
80             sub = dist_matrix[np.ix_(members, members)]
81             new_medoids[ci] = members[np.argmin(sub.sum(axis=1))]
82         if np.array_equal(new_medoids, medoid_idx):
83             break
84         medoid_idx = new_medoids
85     labels = np.argmin(dist_matrix[:, medoid_idx], axis=1)
86     return labels
87
88
89 # --- DIANA (Divisive Analysis, fully vectorized) ---
90 def diana_fit(dist_matrix, k):
91     """Still  $O(n^2)$ - $O(n^3)$  overall, so keep n (pixel count) small."""
92     n = dist_matrix.shape[0]
93     clusters = [np.arange(n)]
94
95     def diameter(idxs):
96         return 0.0 if len(idxs) < 2 else dist_matrix[np.ix_(idxs, idxs)].max()
97
98     while len(clusters) < k:
99         diam = [diameter(c) for c in clusters]
100         split_i = int(np.argmax(diam))
101         cluster = clusters.pop(split_i)
102         if len(cluster) < 2:
103             clusters.append(cluster)
104             break
105
106         sub = dist_matrix[np.ix_(cluster, cluster)]
107         splinter = [cluster[int(np.argmax(sub.mean(axis=1)))]]
108         remaining = list(cluster[cluster != splinter[0]])
109
110         moved = True
111         while moved and len(remaining) > 1:
112             moved = False
113             R, S = np.array(remaining), np.array(splinter)
114             m = len(R)
115             avg_to_remaining = dist_matrix[np.ix_(R, R)].sum(axis=1) / max(m - 1, 1)
116             avg_to_splinter = dist_matrix[np.ix_(R, S)].mean(axis=1)
117             diff = avg_to_remaining - avg_to_splinter
118             idx_max = int(np.argmax(diff))

```

```

119         if diff[idx_max] > 0:
120             remaining.remove(R[idx_max])
121             splinter.append(R[idx_max])
122             moved = True
123
124         clusters.append(np.array(splinter))
125         if remaining:
126             clusters.append(np.array(remaining))
127
128         labels = np.empty(n, dtype=int)
129         for ci, idxs in enumerate(clusters):
130             labels[idxs] = ci
131         return labels
132
133
134 # --- Agglomerative Hierarchical Clustering, 5 linkages ---
135 LINKAGES = ["single", "complete", "average", "ward", "centroid"]
136
137 def agglomerative_fit(X, k, method):
138     Z = linkage(X, method=method, metric="euclidean")
139     labels = fcluster(Z, t=k, criterion="maxclust") - 1 # zero-index
140     return labels
141
142
143 # --- K-Means, DBSCAN, BIRCH via scikit-learn ---
144 def kmeans_fit(X, k):
145     return KMeans(n_clusters=k, n_init=10, random_state=42).fit_predict(X)
146
147 def dbscan_fit(X, eps=0.08, min_samples=8):
148     return DBSCAN(eps=eps, min_samples=min_samples).fit_predict(X)
149
150 def birch_fit(X, k):
151     return Birch(n_clusters=k, threshold=0.3).fit_predict(X)
152
153
154 # --- Internal evaluation + visualization helpers ---
155 def evaluate(X, labels, name, elapsed):
156     mask = labels != -1 # drop DBSCAN noise, if any
157     n_clusters = len(set(labels[mask]))
158     n_noise = int((labels == -1).sum())
159
160     if n_clusters < 2 or mask.sum() < n_clusters:
161         sil, db, ch = np.nan, np.nan, np.nan
162     else:
163         sil = silhouette_score(X[mask], labels[mask])
164         db = davies_bouldin_score(X[mask], labels[mask])
165         ch = calinski_harabasz_score(X[mask], labels[mask])
166
167     return {
168         "Algorithm": name,
169         "Clusters found": n_clusters,
170         "Noise points": n_noise,
171         "Silhouette (higher better)": sil,
172         "Davies-Bouldin (lower better)": db,
173         "Calinski-Harabasz (higher better)": ch,
174         "Time (s)": round(elapsed, 3),
175     }
176

```

```

177 def labels_to_image(labels, img_arr, h, w):
178     """Recolor each cluster with its mean RGB color for a segmentation preview."""
179     flat = img_arr.reshape(-1, 3).astype(float)
180     out = np.zeros_like(flat)
181     for lab in set(labels):
182         mask = labels == lab
183         color = flat[mask].mean(axis=0) if lab != -1 else np.array([255, 0, 0]) # red =
184                                     noise
185         out[mask] = color
186     return out.reshape(h, w, 3).astype(np.uint8)
187
188 # --- Run every algorithm and collect results ---
189 results = []
190 segmented_images = {}
191
192 def run(name, fit_fn):
193     t0 = time.time()
194     labels = fit_fn()
195     elapsed = time.time() - t0
196     results.append(evaluate(X, labels, name, elapsed))
197     segmented_images[name] = labels_to_image(labels, img_arr, h, w)
198     print(f"{name:22s} done in {elapsed:.2f}s")
199
200 run("K-Means", lambda: kmeans_fit(X, K))
201 run("K-Medoids", lambda: kmedoids_fit(dist_matrix, K))
202 run("DBSCAN", lambda: dbscan_fit(X, eps=0.08, min_samples=8))
203 run("DIANA", lambda: diana_fit(dist_matrix, K))
204 for method in LINKAGES:
205     run(f"Agglomerative-{method}", lambda m=method: agglomerative_fit(X, K, m))
206 run("BIRCH", lambda: birch_fit(X, K))
207
208 results_df = pd.DataFrame(results)
209 print(results_df)
210
211 # --- Visualize the segmentations side by side ---
212 n_algos = len(segmented_images)
213 cols = 4
214 rows = int(np.ceil((n_algos + 1) / cols))
215
216 fig, axes = plt.subplots(rows, cols, figsize=(4 * cols, 4 * rows))
217 axes = axes.flatten()
218
219 axes[0].imshow(img_arr); axes[0].set_title("Original"); axes[0].axis("off")
220
221 for i, (name, seg) in enumerate(segmented_images.items(), start=1):
222     axes[i].imshow(seg)
223     axes[i].set_title(name, fontsize=10)
224     axes[i].axis("off")
225
226 for j in range(n_algos + 1, len(axes)):
227     axes[j].axis("off")
228
229 plt.tight_layout()
230 plt.show()
231
232
233

```



```

234 # --- Compare internal metrics across algorithms ---
235 display_df = results_df.sort_values("Silhouette (higher better)", ascending=False).
    reset_index(drop=True)
236 print(display_df)
237
238 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
239 metrics = ["Silhouette (higher better)", "Davies-Bouldin (lower better)", "Calinski-
    Harabasz (higher better)"]
240 colors = ["#4C72B0", "#DD8452", "#55A868"]
241
242 for ax, metric, c in zip(axes, metrics, colors):
243     d = results_df.dropna(subset=[metric]).sort_values(metric, ascending=False)
244     ax.barh(d["Algorithm"], d[metric], color=c)
245     ax.set_title(metric)
246     ax.invert_yaxis()
247
248 plt.tight_layout()
249 plt.show()

```

4 Number of Use Cases and Results

4.1 Use Cases

A **use case** here is defined as one clustering algorithm (or linkage variant) applied to the feature matrix of the sample image. In total, **10 use cases** were executed:

1. K-Means ($K = 6$)
2. K-Medoids – custom PAM-style implementation ($K = 6$)
3. DBSCAN ($\epsilon = 0.08$, $\text{min_samples} = 8$)
4. DIANA (Divisive Analysis) – custom top-down implementation ($K = 6$)
5. Agglomerative Hierarchical Clustering – *single* linkage
6. Agglomerative Hierarchical Clustering – *complete* linkage
7. Agglomerative Hierarchical Clustering – *average* linkage
8. Agglomerative Hierarchical Clustering – *ward* linkage
9. Agglomerative Hierarchical Clustering – *centroid* linkage
10. BIRCH ($K = 6$, $\text{threshold} = 0.3$)

Each use case was scored on the same three internal validity metrics – **Silhouette score** (higher is better), **Davies–Bouldin index** (lower is better), and **Calinski–Harabasz index** (higher is better) – along with running time.

4.2 Quantitative Results

Table 1 lists the results for all 10 use cases, sorted by Silhouette score in descending order.

Table 1: Clustering evaluation results on the sample image ($K = 6$ target clusters).

Algorithm	Clusters	Noise	Silhouette $\uparrow$	Davies–Bouldin $\downarrow$	Calinski–Harabasz $\uparrow$	Time (s)
K-Means	6	0	0.4058	0.8938	773.24	0.080
K-Medoids	6	0	0.4018	0.8995	765.72	0.007
Agglomerative – average	6	0	0.3798	0.9712	689.77	0.022
DIANA	6	0	0.3796	0.9524	710.56	2.696
Agglomerative – complete	6	0	0.3707	0.9313	699.11	0.020
Agglomerative – ward	6	0	0.3587	0.9708	644.49	0.026
Agglomerative – centroid	6	0	0.3503	0.9281	642.69	0.025
DBSCAN	3	1	0.2948	1.0156	440.98	0.015
Agglomerative – single	6	0	0.0688	1.4081	183.90	0.010
BIRCH	1	0	–	–	–	0.023

Observations:

- **K-Means** and **K-Medoids** produce the best-separated, most compact clusters (highest Silhouette, lowest Davies–Bouldin, highest Calinski–Harabasz), confirming that the colour+spatial feature space is close to convex and roughly isotropic – exactly the geometry these centroid/medoid-based methods are designed for.
- Among the five **Agglomerative** linkages, *average* and *complete* linkage come closest to K-Means quality, while *single* linkage performs far worse (see Section 5).
- The custom-built **DIANA** implementation is competitive with the Agglomerative variants in quality, but is roughly $100\times$ slower because both DIANA and K-Medoids operate on the full 1024×1024 pairwise distance matrix.
- **DBSCAN** recovers only 3 of the requested 6 clusters, since density-based clustering does not take K as a direct input.
- **BIRCH** collapses to a single cluster under the chosen threshold, so validity metrics are undefined (NaN) – discussed further in Section 5.

4.3 Visual Results

Figure 2 shows the segmented output of every algorithm, recoloured so that each pixel takes the mean RGB colour of its assigned cluster. Figure 3 compares the three validity metrics across all algorithms as bar charts.

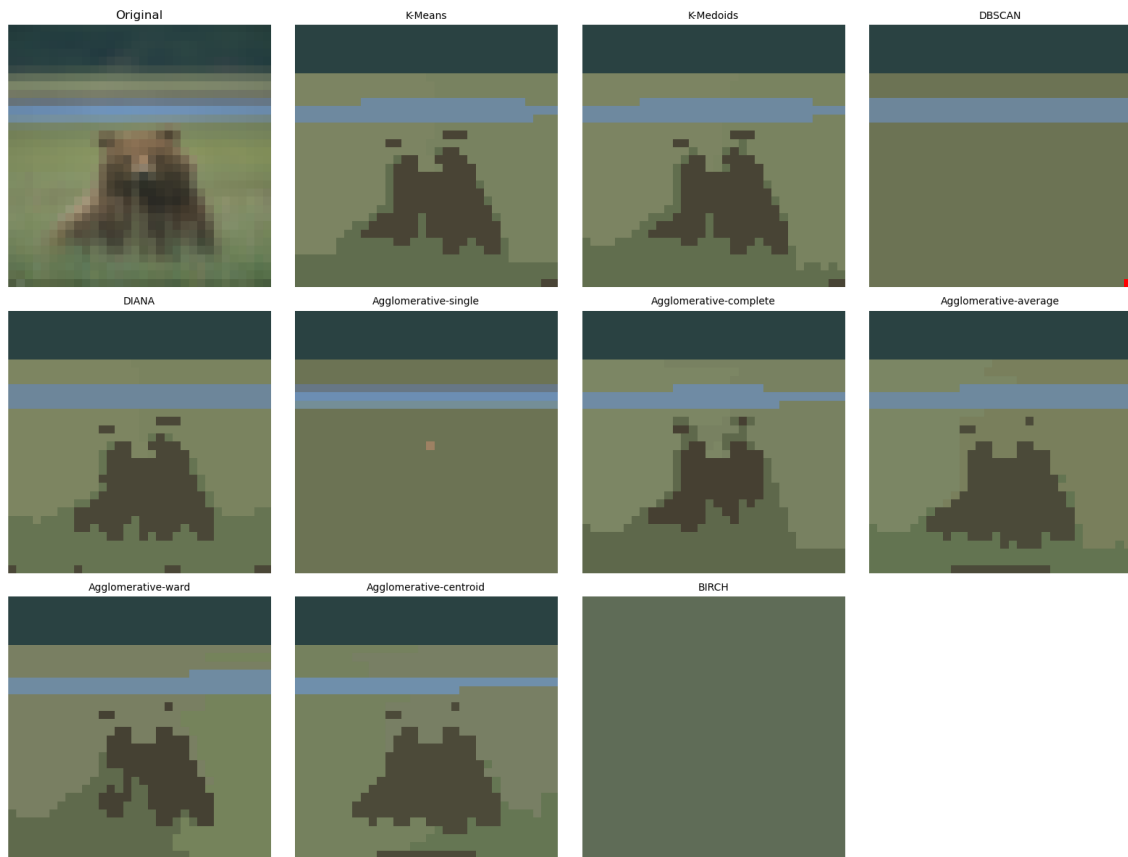


Figure 2: Segmentation output of all 10 use cases compared against the original image.

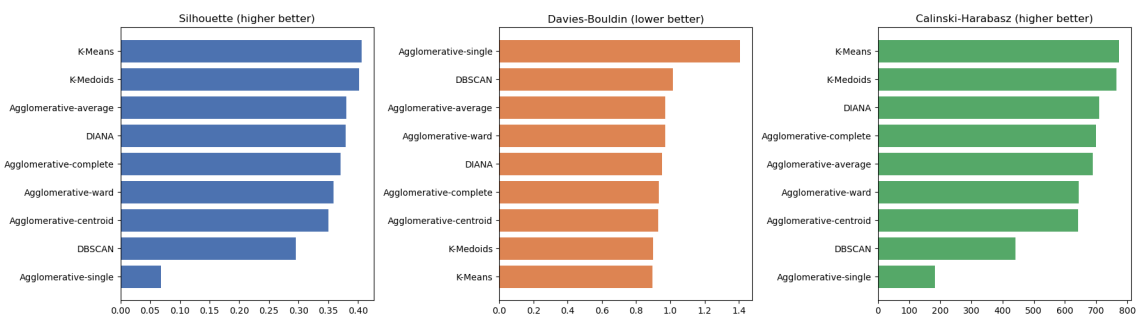


Figure 3: Silhouette, Davies–Bouldin, and Calinski–Harabasz scores compared across all algorithms.

5 Failure Scenarios

Three of the ten use cases illustrate common, instructive failure modes of clustering algorithms when applied to image segmentation. The corresponding output crops are shown in Figure 4, with a successful K-Means segmentation included alongside them for reference.

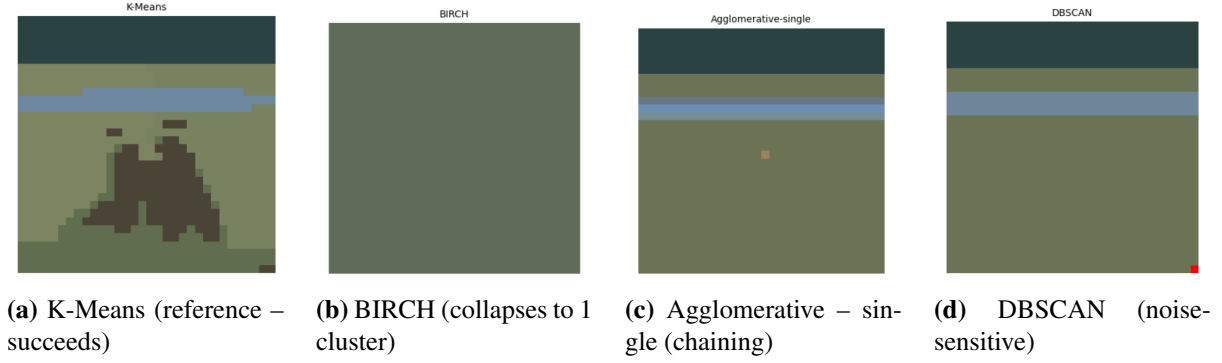


Figure 4: A correctly segmented reference output next to the three failure scenarios.

1. **BIRCH – under-segmentation (collapse to one cluster).** With `threshold=0.3`, all 1024 pixels are absorbed into a single CF-subcluster, so BIRCH returns only 1 cluster instead of the requested 6 (scikit-learn raises a `ConvergenceWarning`: “Number of subclusters found (1) ... is less than (6). Decrease the threshold.”). Because a minimum of 2 clusters is required to compute Silhouette, Davies–Bouldin, or Calinski–Harabasz, all three metrics are undefined (NaN) for this run. *Root cause:* the BIRCH threshold defines the maximum radius of a sub-cluster in feature space; on this compact, low-variance 5-D feature space the chosen threshold is larger than the spread of the entire dataset.
2. **Agglomerative (single linkage) – chaining effect.** Single linkage merges clusters based on their *closest* pair of points, which makes it highly susceptible to the classic *chaining effect*: a thin bridge of intermediate pixels causes almost the entire image to be absorbed into one dominant cluster, leaving the remaining “clusters” as isolated single pixels or tiny fragments (visible in Figure 4(c) as a single stray coloured pixel against an otherwise uniform background). This is reflected quantitatively as the worst Silhouette score among all runs that returned ≥ 2 clusters (0.0688) and the worst Davies–Bouldin index (1.4081).
3. **DBSCAN – density/parameter sensitivity.** DBSCAN does not accept the target cluster count K as a parameter; it infers clusters purely from local density (ϵ and `min_samples`), so it recovered only 3 clusters instead of 6 and flagged low-density pixels as noise (label -1 , rendered in red in Figure 2). Re-running the identical code with the same parameters on a different machine/library version changed the number of noise points from 1 to 45, and the Silhouette score dropped from 0.29 to near 0.00 – demonstrating that DBSCAN’s outcome is highly sensitive to the exact values of ϵ and `min_samples` relative to the density of the feature space, and does not generalise well without per-image parameter tuning.

6 Conclusion

Across all 10 use cases, centroid/medoid-based methods (**K-Means** and **K-Medoids**) gave the best and most stable segmentations of the BSDS sample image on every internal validity metric, while remaining extremely fast. **Average** and **complete**-linkage Agglomerative clustering and the custom **DIANA** implementation were close runners-up, at the cost of significantly higher run-time for DIANA. In contrast, **single**-linkage Agglomerative clustering, **BIRCH** (with an unsuitable threshold), and **DBSCAN** (with un-tuned parameters) each illustrate a distinct, well-known failure mode of clustering – chaining, threshold under-segmentation, and density-based noise sensitivity, respectively. This confirms that algorithm choice and hyperparameter tuning matter as much as the feature representation itself when clustering is used for image segmentation.